AI-Powered Requirement Gathering & Documentation System

Development Build Guide — FastAPI + React Edition

Xccelera AI Team | Version 2.0 | June 2026

*This guide follows a **development-first approach**: build and validate the full system locally using FastAPI, React, PostgreSQL, and Docker — no cloud account required during development. GCP infrastructure is introduced only in the final phase as a migration step once the system is proven and ready for production.*

> **Note: This is Version 2.0 of the build guide, updated per team direction.**
> GCP is not mandatory for development. All phases (1–7) run entirely on your local machine or a simple VPS.
> Phase 8 covers GCP migration — only follow it when the system is ready for production launch.

Overview & Prerequisites

This guide uses a simple, cloud-free stack during development. You build on your local machine using tools every developer already has — Docker, Python, and Node.js. The system is validated fully before any cloud infrastructure is involved.

What You Are Building

A SaaS platform that:

- Accepts client inputs in any format — video, audio, documents, email, chat, or typed notes.
- Automatically transcribes, processes, and extracts structured requirements from those inputs using AI.
- Generates a fully source-cited, completeness-scored Project Requirements Document (PRD).
- Provides a secure client review portal for comments, follow-ups, and click-to-approve workflow.
- Runs a Feasibility Agent that checks sanctions lists and regulations, auto-injecting compliance requirements.

Development Stack (No Cloud Required)

Layer	Dev Technology	Production Equivalent (Phase 8)
Backend API	Python 3.12 + FastAPI	Same — deployed to Cloud Run
Database	PostgreSQL 15 (Docker)	Cloud SQL (PostgreSQL 15)
File Storage	Local filesystem / MinIO (Docker)	GCP Cloud Storage
Async Queue	Celery + Redis (Docker)	Google Cloud Pub/Sub
AI / LLM	Gemini API (direct HTTP) or OpenAI	Vertex AI (Gemini 1.5 Pro)
Transcription	Whisper (local Python library)	Whisper on Cloud Run
Auth	FastAPI JWT (simple)	Firebase Authentication
Frontend	React 18 + Tailwind CSS (Vite)	Same — served via Cloud CDN
Infrastructure	Docker Compose	Terraform on GCP
CI/CD	Local scripts / GitHub Actions	GitHub Actions + Cloud Build

Team Roles Required

Role	Responsibilities	Phase Involved
Backend Engineer (x2)	FastAPI, AI pipeline, database, async jobs	All phases
Frontend Engineer (x1)	React + Tailwind, client portal, export UI	Phase 4, 5
AI/ML Engineer (x1)	Whisper, Gemini API, embeddings, gap analysis	Phase 2, 3

Role	Responsibilities	Phase Involved
QA Engineer (x1)	End-to-end testing, load testing, UAT	Phase 6
DevOps / Infra Engineer (x1)	Docker Compose, CI/CD, Phase 8 GCP migration	Phase 1, 8
Product / BA Lead (x1)	Requirements validation, internal UAT	Phase 6, 7

Local Developer Machine Setup

Every developer needs the following — no cloud accounts required to start:

Tool	Version	Purpose
Python	3.12+	Backend runtime
Node.js	20 LTS+	Frontend tooling (Vite + React)
Docker Desktop	Latest	Runs PostgreSQL, Redis, MinIO locally
Git	2.40+	Version control
VS Code	Latest	Recommended editor
Postman or Bruno	Latest	API testing
ffmpeg	Latest	Required by Whisper for audio processing

> **Tip: All backing services (PostgreSQL, Redis, MinIO) run via Docker Compose.**
> Developers only need Python and Node.js installed natively — everything else is containerised.
> Run: docker compose up -d to start all services in one command.

Phase 1 — Local Development Environment (Day 1)

Set up the complete local development environment using Docker Compose. By the end of this phase, every developer has a running database, file storage, and message queue on their machine.

Step 1.1 — Repository Structure

Create this folder layout at the root of the repository:

```
xccelera-prd/
  backend/          # FastAPI application
  frontend/         # React application
  workers/          # Celery background workers
  docker-compose.yml
  docker-compose.prod.yml  # Used in Phase 8 only
  .env.example      # Template — copy to .env and fill in
  README.md
```

Step 1.2 — Docker Compose (All Local Services)

Create docker-compose.yml at the project root. This starts PostgreSQL, Redis, and MinIO (local S3-compatible file storage):

```
version: "3.9"
services:

  postgres:
    image: pgvector/pgvector:pg15
    environment:
      POSTGRES_USER: prd_user
      POSTGRES_PASSWORD: prd_pass
      POSTGRES_DB: prd_db
    ports: ["5432:5432"]
    volumes: ["postgres_data:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    ports: ["9000:9000", "9001:9001"]
    volumes: ["minio_data:/data"]

volumes:
  postgres_data:
  minio_data:
```

Start all services:

```
docker compose up -d

# Verify all three are running:
docker compose ps

# MinIO web console: http://localhost:9001  (admin / admin)
# PostgreSQL:        localhost:5432
# Redis:             localhost:6379
```

Step 1.3 — Environment Variables

Copy .env.example to .env and fill in the values. During development, all values are local:

```
# .env  (development)
DATABASE_URL=postgresql+asyncpg://prd_user:prd_pass@localhost:5432/prd_db
REDIS_URL=redis://localhost:6379/0

# MinIO (local S3-compatible storage)
STORAGE_ENDPOINT=http://localhost:9000
STORAGE_ACCESS_KEY=minioadmin
STORAGE_SECRET_KEY=minioadmin
STORAGE_BUCKET=prd-uploads

# AI Keys — get from Gemini or OpenAI console
GEMINI_API_KEY=your_gemini_api_key_here
OPENAI_API_KEY=your_openai_key_here  # optional fallback

# Auth
SECRET_KEY=dev-secret-change-in-production-32chars
ACCESS_TOKEN_EXPIRE_MINUTES=1440

# Email (use Mailtrap for dev — free, catches all outgoing emails)
SMTP_HOST=sandbox.smtp.mailtrap.io
SMTP_PORT=2525
SMTP_USER=your_mailtrap_user
SMTP_PASS=your_mailtrap_pass
```

> **Tip: Use Mailtrap (mailtrap.io) for email during development — it catches all sent emails**
> in a sandbox inbox so you can test notifications without sending real emails.
> Replace with SendGrid credentials only in Phase 8 (production).

Step 1.4 — Create MinIO Bucket

After MinIO is running, create the upload bucket using the MinIO CLI:

```
# Install MinIO client
pip install minio

# Or use the mc CLI:
mc alias set local http://localhost:9000 minioadmin minioadmin
mc mb local/prd-uploads
mc mb local/prd-exports

# Verify:
mc ls local
```

Phase 2 — Backend: FastAPI Application (Week 1)

Build the core FastAPI backend. This phase delivers all REST API endpoints, database models, JWT authentication, and file upload handling — running entirely locally.

Step 2.1 — Project Structure (Backend)

```
backend/
  app/
    main.py               # FastAPI entry point + app config
    api/
      routes/
        auth.py           # Login, register, token refresh
        projects.py       # Project CRUD
        uploads.py        # File upload endpoints
        prd.py            # PRD generation & retrieval
        feasibility.py    # Feasibility agent trigger
        approval.py       # Client approval workflow
    core/
      config.py           # Settings loaded from .env
      security.py         # JWT creation & verification
      database.py         # SQLAlchemy async engine
    models/               # SQLAlchemy ORM table models
    schemas/              # Pydantic request/response schemas
    services/
      storage.py          # MinIO / S3 file operations
      queue.py            # Celery task dispatch
      transcription.py    # Whisper wrapper
      extraction.py       # Gemini entity extraction
      prd_engine.py       # PRD assembly logic
      gap_analysis.py     # Completeness scoring
      feasibility.py      # Feasibility agent
      export.py           # PDF / Word export
      email.py            # Email notifications
  requirements.txt
  Dockerfile
  alembic/               # Database migrations
```

Step 2.2 — Python Dependencies

Create requirements.txt:

```
# Core
fastapi==0.111.0
uvicorn[standard]==0.30.0
python-multipart==0.0.9     # file uploads

# Database
sqlalchemy[asyncio]==2.0.30
asyncpg==0.29.0
alembic==1.13.0
pgvector==0.2.5
```

```
# Auth
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4

# Storage (MinIO / S3)
boto3==1.34.0              # works with MinIO via S3-compatible API

# Queue
celery==5.4.0
redis==5.0.0

# AI
google-generativeai==0.7.0  # Gemini API (direct, no Vertex AI needed)
openai==1.30.0              # fallback
openai-whisper==20231117    # local transcription
numpy==1.26.4

# PDF export
weasyprint==61.0
jinja2==3.1.4

# Word export
python-docx==1.1.0

# Utilities
python-dotenv==1.0.0
httpx==0.27.0
aiofiles==23.2.1
```

Step 2.3 — FastAPI App Entry Point

```
# app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.routes import auth, projects, uploads, prd, feasibility, approval
from app.core.database import create_tables

app = FastAPI(title="AI PRD System", version="2.0.0")

app.add_middleware(
  CORSMiddleware,
  allow_origins=["http://localhost:5173"],  # Vite dev server
  allow_methods=["*"],
  allow_headers=["*"],
)

app.include_router(auth.router,        prefix="/api/auth")
app.include_router(projects.router,    prefix="/api/projects")
app.include_router(uploads.router,     prefix="/api/projects")
app.include_router(prd.router,         prefix="/api/prd")
app.include_router(feasibility.router, prefix="/api/feasibility")
app.include_router(approval.router,    prefix="/api/approval")

@app.get("/api/health")
async def health(): return {"status": "ok"}

@app.on_event("startup")
async def startup(): await create_tables()
```

Step 2.4 — JWT Authentication (No Firebase Needed)

For development, use a simple JWT-based auth system built into FastAPI:

```
# app/core/security.py
from jose import jwt
from passlib.context import CryptContext
from datetime import datetime, timedelta
from app.core.config import settings

pwd_context = CryptContext(schemes=["bcrypt"])

def create_access_token(user_id: str, role: str) -> str:
  payload = {
    "sub": user_id,
    "role": role,
    "exp": datetime.utcnow() + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
  }
  return jwt.encode(payload, settings.SECRET_KEY, algorithm="HS256")

def verify_token(token: str) -> dict:
  return jwt.decode(token, settings.SECRET_KEY, algorithms=["HS256"])

def hash_password(password: str) -> str:
  return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
  return pwd_context.verify(plain, hashed)
```

Step 2.5 — Database Schema

Run these via Alembic migrations. The schema is identical to the GCP version — only the connection string changes.

```
-- Users table (replaces Firebase Auth for dev)
CREATE TABLE users (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email      TEXT UNIQUE NOT NULL,
  password   TEXT NOT NULL,   -- bcrypt hash
  role       TEXT DEFAULT 'BA_PM',  -- Admin | BA_PM | Client_Reviewer
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE projects (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name         TEXT NOT NULL,
  client_name  TEXT,
  client_email TEXT,
  status       TEXT DEFAULT 'active',
  created_by   UUID REFERENCES users(id),
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  updated_at   TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE source_files (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID REFERENCES projects(id) ON DELETE CASCADE,
  file_name   TEXT NOT NULL,
  file_type   TEXT NOT NULL,   -- video | audio | document | chat | text
  storage_key TEXT NOT NULL,   -- MinIO object key (dev) / GCS path (prod)
```

```
  status      TEXT DEFAULT 'pending',
  transcript  TEXT,
  uploaded_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE requirements (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id       UUID REFERENCES projects(id) ON DELETE CASCADE,
  req_id           TEXT,
  req_type         TEXT,
  description      TEXT NOT NULL,
  confidence_score FLOAT,
  section          TEXT,
  injected_by      TEXT,
  embedding        vector(768),
  created_at       TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE requirement_sources (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  requirement_id UUID REFERENCES requirements(id) ON DELETE CASCADE,
  source_file_id UUID REFERENCES source_files(id),
  location       TEXT
);
```

```
CREATE TABLE prd_versions (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id     UUID REFERENCES projects(id),
  version_number INT DEFAULT 1,
  content_json   JSONB,
  export_path    TEXT,       -- MinIO key (dev) / GCS path (prod)
  language       TEXT DEFAULT 'en',
  created_at     TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE approvals (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  prd_version_id UUID REFERENCES prd_versions(id),
  approver_name  TEXT,
  approver_email TEXT,
  approved_at    TIMESTAMPTZ DEFAULT NOW()
);
```

Step 2.6 — File Upload Endpoint

```
# app/api/routes/uploads.py
@router.post("/{project_id}/files")
async def upload_file(
  project_id: UUID,
  file: UploadFile,
  background_tasks: BackgroundTasks,
  current_user = Depends(get_current_user),
  db: AsyncSession = Depends(get_db)
):
  validate_file(file)  # check type & size

  # Upload to MinIO (dev) — same boto3 API as S3/GCS in prod
  storage_key = await storage_service.upload(file, str(project_id))
```

```
  source_file = SourceFile(
    project_id=project_id,
    file_name=file.filename,
    file_type=detect_file_type(file.filename),
    storage_key=storage_key
  )
  db.add(source_file)
  await db.commit()

  # Enqueue transcription as a Celery task
  transcribe_file.delay(str(source_file.id), storage_key)

  return {'id': source_file.id, 'status': 'processing'}
```

Step 2.7 — Celery Worker Setup (Replaces Pub/Sub)

Celery + Redis handles all background processing during development. Same logic as Pub/Sub workers — only the transport layer differs.

```
# workers/celery_app.py
from celery import Celery
import os

app = Celery(
  'prd_workers',
  broker=os.getenv('REDIS_URL', 'redis://localhost:6379/0'),
  backend=os.getenv('REDIS_URL', 'redis://localhost:6379/0'),
)

app.conf.task_routes = {
  'workers.tasks.transcribe_file':    {'queue': 'transcription'},
  'workers.tasks.extract_requirements': {'queue': 'extraction'},
  'workers.tasks.generate_prd':        {'queue': 'prd'},
}
```

```
# workers/tasks.py
from workers.celery_app import app

@app.task(bind=True, max_retries=3)
def transcribe_file(self, source_file_id: str, storage_key: str):
  try:
    result = transcription_service.run(storage_key)
    db.update_source_file(source_file_id, transcript=result["text"], status="done")
    # Trigger next step
    extract_requirements.delay(source_file_id)
  except Exception as exc:
    raise self.retry(exc=exc, countdown=30)

@app.task
def extract_requirements(source_file_id: str):
  transcript = db.get_transcript(source_file_id)
  requirements = extraction_service.run(transcript, source_file_id)
  db.save_requirements(requirements)
  check_and_generate_prd.delay(source_file_id)

@app.task
def generate_prd(project_id: str):
  requirements = db.get_all_requirements(project_id)
  prd = prd_engine.generate(requirements)
```

```
    db.save_prd_version(project_id, prd)
```

Start the workers in a separate terminal:

```
# Terminal 1 — API server
uvicorn app.main:app --reload --port 8000

# Terminal 2 — Transcription worker
celery -A workers.celery_app worker -Q transcription -c 2

# Terminal 3 — Extraction + PRD workers
celery -A workers.celery_app worker -Q extraction,prd -c 4
```

Phase 3 — AI Pipeline: Whisper + Gemini (Week 2)

Build the intelligence core using local Whisper for transcription and direct Gemini API calls for extraction and PRD generation. No Vertex AI account needed during development.

Step 3.1 — Whisper Transcription (Local)

Whisper runs as a Python library — no Docker, no cloud API. Install it alongside the backend:

```
pip install openai-whisper
pip install ffmpeg-python   # Python wrapper for ffmpeg

# Test it works:
python -c "import whisper; m = whisper.load_model('base'); print('OK')"
```

```
# services/transcription.py
import whisper
import tempfile
from app.services.storage import storage_service

model = whisper.load_model("medium")  # base | small | medium | large

async def transcribe(storage_key: str) -> dict:
  # Download file from MinIO to a temp file
  with tempfile.NamedTemporaryFile(suffix=".tmp", delete=False) as tmp:
    await storage_service.download(storage_key, tmp.name)
    result = model.transcribe(tmp.name, word_timestamps=True)

  segments = [
    {
      'text':           seg['text'],
      'start':          seg['start'],
      'end':            seg['end'],
      'low_confidence': seg.get('avg_logprob', 0) < -1.0
    }
    for seg in result['segments']
  ]
  return {'full_text': result['text'], 'segments': segments}
```

> Whisper model sizes — choose based on your machine:
> base — fast, lower accuracy, good for testing (74MB)
> small — good balance for development (244MB)
> medium — recommended for real requirements work (769MB)
> large — highest accuracy, needs 10GB+ RAM (1550MB)

Step 3.2 — PII Redaction Before Storage

```
# services/transcription.py (continued)
import re

def redact_pii(text: str) -> str:
```

```
  text = re.sub(r'[\w.+-]+@[\w-]+\.[\w.]+', '[EMAIL]', text)
  text = re.sub(r'(\+91|0)?[6-9]\d{9}', '[PHONE]', text)
  text = re.sub(r'\b\d{4}\s?\d{4}\s?\d{4}\b', '[ID]', text)
  return text
```

Step 3.3 — Gemini API (Direct — No Vertex AI)

Use the google-generativeai SDK which calls the Gemini API directly with just an API key. Get a free API key from Google AI Studio (aistudio.google.com).

```
# services/extraction.py
import google.generativeai as genai
from app.core.config import settings
import json

genai.configure(api_key=settings.GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-1.5-pro")

EXTRACTION_PROMPT = """
You are a requirements analyst. Read the following text and extract ALL
functional requirements, non-functional requirements, constraints, timeline
mentions, and stakeholder references.

For each item return a JSON object with:
  - type: functional | non_functional | constraint | timeline | stakeholder
  - description: the requirement in clear, formal language
  - confidence: 0.0 to 1.0
  - quote: exact phrase from the text supporting this requirement

Return ONLY a valid JSON array. No explanation. No preamble.

Text: {chunk}
"""

async def extract_requirements(chunk: str, source_ref: str) -> list[dict]:
  response = model.generate_content(EXTRACTION_PROMPT.format(chunk=chunk))
  items = json.loads(response.text)
  for item in items: item['source_ref'] = source_ref
  return items
```

> Why direct Gemini API and not Vertex AI?
> The google-generativeai SDK works with a single API key — no GCP project, no service account, no IAM configuration. Identical model quality to Vertex AI during development.
> In Phase 8 (GCP migration), swap the SDK to vertexai — the prompt code is unchanged.

Step 3.4 — Text Chunking & Embedding

```
def chunk_text(text: str, max_tokens: int = 800) -> list[str]:
  sentences = text.split('. ')
  chunks, current = [], ''
  for sentence in sentences:
    if len(current) + len(sentence) > max_tokens:
      chunks.append(current)
      current = sentence
    else:
      current += sentence + '. '
  if current: chunks.append(current)
```

```
  return chunks

# Embeddings via Gemini embedding model (free tier available)
async def embed_text(text: str) -> list[float]:
  result = genai.embed_content(model='models/embedding-001', content=text)
  return result['embedding']
```

Step 3.5 — Deduplication, Gap Analysis & PRD Generation

These are identical to the GCP version — the logic is pure Python with no cloud dependency.

```
# services/gap_analysis.py
PRD_SECTIONS = {
  'project_overview':           ['goal', 'purpose', 'background'],
  'business_objectives':        ['metric', 'KPI', 'success'],
  'stakeholders':               ['user', 'persona', 'role'],
  'scope':                      ['in scope', 'out of scope'],
  'functional_requirements':    ['shall', 'must', 'should', 'feature'],
  'non_functional_requirements':['performance', 'security', 'availability'],
  'user_stories':               ['as a user', 'I want', 'so that'],
  'technical_constraints':      ['technology', 'integration', 'API'],
  'data_requirements':          ['data', 'storage', 'database'],
  'timeline':                   ['deadline', 'milestone', 'launch', 'week'],
  'assumptions':                ['assume', 'dependency', 'depend'],
  'open_questions':             [],
  'glossary':                   [],
  'source_index':               [],
}

def score_section(section_key: str, requirements: list[dict]) -> float:
  keywords = PRD_SECTIONS[section_key]
  if not keywords: return 0.0
  matched = sum(1 for r in requirements for k in keywords if k in r['description'].lower())
  return min(matched / (len(keywords) * 2), 1.0)
```

```
# services/prd_engine.py
PRD_PROMPT = """
You are a professional Business Analyst. Using the structured requirements
provided, generate a complete PRD following the 14-section template.
Each requirement must include its source citation in brackets.

Requirements: {requirements_json}
Completeness scores: {scores_json}

For sections with score below 0.4, add a clear placeholder:
[INCOMPLETE — see Open Questions section]
Return as structured JSON matching the 14-section schema.
"""

async def generate_prd(project_id: UUID) -> dict:
  requirements = await db.get_requirements(project_id)
  scores = {k: score_section(k, requirements) for k in PRD_SECTIONS}
  response = model.generate_content(
    PRD_PROMPT.format(
      requirements_json=json.dumps(requirements),
      scores_json=json.dumps(scores)
    )
  )
  return json.loads(response.text)
```

Phase 4 — Frontend: React Application (Week 3)

Build the React frontend using Vite as the dev server. The frontend talks to the FastAPI backend via REST. No special cloud SDK needed — just standard fetch/axios calls.

Step 4.1 — Project Setup (Vite + React + Tailwind)

```
# Scaffold the React app
npm create vite@latest frontend -- --template react
cd frontend
npm install

# Add Tailwind CSS
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Add routing and HTTP client
npm install react-router-dom axios

# Add useful UI utilities
npm install @headlessui/react react-dropzone react-hot-toast
```

Step 4.2 — Project Structure (Frontend)

```
frontend/src/
  components/
    layout/      # Sidebar, TopBar, AppShell
    projects/    # ProjectCard, ProjectList, CreateProjectModal
    upload/      # FileDropzone, UploadProgress, SourceFileList
    prd/         # PRDViewer, SectionNav, CitationBadge, CompletenessBar
    review/      # CommentThread, FollowUpPanel, ApprovalBanner
    feasibility/ # FeasibilityPanel, ScoreBadge
    export/      # ExportButton
    common/      # Button, Modal, Badge, Spinner, Toast
  pages/
    Dashboard.jsx
    ProjectDetail.jsx
    ClientPortal.jsx
    Login.jsx
  hooks/
    useAuth.js
    useProject.js
    usePRD.js
  services/
    api.js         # axios instance + all API calls
  context/
    AuthContext.jsx
  App.jsx
  main.jsx
```

Step 4.3 — API Service Layer

```
// src/services/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000',
});

// Attach JWT to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem('access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Auth
export const login = (email, password) =>
  api.post('/api/auth/login', { email, password });

// Projects
export const getProjects  = () => api.get('/api/projects');
export const createProject = (data) => api.post('/api/projects', data);
export const getProject   = (id) => api.get(`/api/projects/${id}`);

// Uploads
export const uploadFile = (projectId, file) => {
  const form = new FormData();
  form.append('file', file);
  return api.post(`/api/projects/${projectId}/files`, form);
};

// PRD
export const getPRD        = (id) => api.get(`/api/prd/${id}`);
export const exportPDF     = (id) => api.get(`/api/prd/${id}/export/pdf`, { responseType: "blob" });

// Feasibility
export const runFeasibility = (projectId) => api.post(`/api/feasibility/${projectId}`);

// Approval
export const approvePRD = (prdId) => api.post(`/api/approval/${prdId}/approve`);
```

Step 4.4 — Environment Variables (Frontend)

```
# frontend/.env
VITE_API_URL=http://localhost:8000

# frontend/.env.production  (filled in during Phase 8)
# VITE_API_URL=https://api.your-production-domain.com
```

Step 4.5 — Key UI Components

Component	Description	Key Behaviour
FileDropzone	Drag-and-drop upload area	Accepts 7 file types; shows per-file progress; polls /status every 5s
PRDViewer	Full PRD document renderer	14 sections in sidebar nav; citation badges on each requirement

Component	Description	Key Behaviour
CitationBadge	Source reference tag	Shows file name + timestamp; click to jump to Source Index
CompletenessBar	Section score indicator	Green >80%, Amber 40–79%, Red <40%
FollowUpPanel	Clarification questions list	Each question has Send to Client button; shows answer when received
ApprovalBanner	Client approval CTA	Shows only in Client Portal; Approve triggers confirmation modal
FeasibilityBadge	Green / Amber / Red score pill	Click to expand full feasibility report
ExportButton	PDF / Word download	GET /api/prd/{id}/export/pdf or /word — triggers file download

Step 4.6 — Client Review Portal

Accessible via /client/review/:project_id — simpler UI than the BA dashboard:

- Client logs in with email/password (account created by BA/PM during project setup)
- Shows: full PRD, completeness scores, follow-up questions from the BA/PM
- Client can comment on sections but cannot edit PRD text directly
- 'Approve' button shows modal: 'I confirm I have reviewed this document and approve it as final.'
- After approval: document locked, read-only banner displayed, BA/PM notified by email

Step 4.7 — PDF & Word Export (Server-Side)

```
# app/services/export.py
from weasyprint import HTML
from jinja2 import Environment, FileSystemLoader
from docx import Document as WordDoc

jinja = Environment(loader=FileSystemLoader("templates/"))

async def export_pdf(prd_version_id: UUID) -> bytes:
  prd = await db.get_prd_version(prd_version_id)
  html_str = jinja.get_template('prd_export.html').render(prd=prd)
  return HTML(string=html_str).write_pdf()

async def export_word(prd_version_id: UUID) -> bytes:
  prd = await db.get_prd_version(prd_version_id)
  doc = WordDoc()
  for section in prd['sections']:
    doc.add_heading(section['title'], level=1)
    for req in section['requirements']:
      doc.add_paragraph(f"{req['req_id']}: {req['description']}")
  buf = io.BytesIO()
  doc.save(buf)
  return buf.getvalue()
```

Phase 5 — Feasibility Agent (Week 3, Parallel)

The feasibility agent performs live web searches and compliance checks. During development it calls external search APIs directly — no GCP dependency.

Step 5.1 — Agent Architecture

The agent uses a tool-calling pattern with Gemini — identical to the GCP version. Only the model initialisation differs (direct API key vs. Vertex AI).

```
# services/feasibility.py
import google.generativeai as genai
import httpx

SYSTEM_PROMPT = """
You are a compliance and geopolitical risk analyst.
You have access to a web_search tool.

Given: country={country}, industry={industry}, project_type={project_type}

1. Check relevant sanctions lists for the country
2. Identify data protection laws (GDPR, PDPA, IT Act, etc.)
3. Check industry-specific compliance requirements
4. Identify export control regulations if relevant (ITAR, EAR)

Return JSON: { overall_score, hard_blockers[], soft_warnings[], compliance_requirements[] }
"""

async def run_feasibility(project_id: UUID) -> dict:
  project = await db.get_project(project_id)
  prompt = SYSTEM_PROMPT.format(
    country=project.client_country,
    industry=project.industry,
    project_type=project.project_type
  )
  model = genai.GenerativeModel('gemini-1.5-pro')
  response = model.generate_content(prompt)
  return json.loads(response.text)
```

Step 5.2 — Sanctions Check Sources

Sanctions List	Source	Trigger Condition
OFAC SDN List	home.treasury.gov/policy-issues/financial-sanctions	US-adjacent project or USD transactions
UN Security Council	scsanctions.un.org	All international projects
EU CFSP List	eur-lex.europa.eu/sanctions	EU clients or SEPA payments
UK OFSI List	assets.publishing.service.gov.uk	UK clients or GBP transactions

Step 5.3 — Auto-Injection into PRD

Compliance requirements identified by the agent are automatically added to PRD Section 6:

```
async def inject_compliance_requirements(project_id: UUID, items: list):
  for item in items:
    req = Requirement(
      project_id=project_id,
      req_type='non_functional',
      description=item['requirement'],
      confidence_score=0.95,
      section='non_functional_requirements',
      injected_by='FeasibilityAgent'
    )
    db.add(req)
    db.add(RequirementSource(
      requirement=req,
      location=f"Feasibility Agent | {item['regulation']}"
    ))
  await db.commit()
```

Important: Each injected requirement shows an 'Accept / Edit / Remove' button in the PRD editor.

BA/PM must review every injected item before PRD can be submitted for client approval.

Hard blockers (RED) prevent submission until resolved or overridden by Admin.

Phase 6 — Testing & QA (Week 4)

All testing runs locally against the Docker Compose environment. No cloud infrastructure needed for any test tier.

Step 6.1 — Unit Tests (pytest)

- test_chunking.py — verify chunks respect sentence boundaries and token limits
- test_extraction.py — mock Gemini responses, verify requirement objects are created correctly
- test_deduplication.py — verify cosine similarity merging with synthetic embeddings
- test_gap_analysis.py — verify scoring produces correct Green/Amber/Red classifications
- test_pii_redaction.py — test with known email, phone, and Aadhaar patterns
- test_export.py — verify PDF and Word output are valid non-empty files
- test_auth.py — verify JWT creation, expiry, and role enforcement

```
# Run all unit tests
pytest backend/tests/unit -v

# With coverage report
pytest backend/tests/unit --cov=app --cov-report=html
open htmlcov/index.html
```

Step 6.2 — Integration Tests

Run the full pipeline against your local Docker Compose stack:

1. Start all services: docker compose up -d && uvicorn app.main:app --reload
2. Upload a sample MP4 file (5-minute mock Zoom recording) via POST /api/projects/{id}/files
3. Verify record in source_files table has status 'pending'
4. Verify Celery picks up the transcription task (check Celery logs)
5. Verify status updates to 'done' and transcript field is populated
6. Verify extraction creates requirement rows in the requirements table
7. Verify PRD generation produces a 14-section content_json document
8. Verify completeness scores are in the 0.0–1.0 range per section

```
# Automated integration test script
pytest backend/tests/integration -v --timeout=120

# The integration tests require docker compose to be running:
docker compose up -d postgres redis minio
```

Step 6.3 — Load Testing (Locust)

```
# locustfile.py
from locust import HttpUser, task, between
```

```
class PRDUser(HttpUser):
  wait_time = between(1, 3)

  def on_start(self):
    r = self.client.post('/api/auth/login', json={'email':'test@test.com','password':'test'})
    self.token = r.json()['access_token']
    self.client.headers = {'Authorization': f'Bearer {self.token}'}

  @task
  def upload_file(self):
    with open('test_recording.mp4', 'rb') as f:
      self.client.post('/api/projects/test-id/files', files={'file': f})

  @task(3)
  def view_prd(self):
    self.client.get('/api/prd/test-prd-id')
```

```
# Run load test:
locust -f locustfile.py --host=http://localhost:8000 --users=50 --spawn-rate=5
# Open http://localhost:8089 to see the Locust dashboard
```

Step 6.4 — Security Testing

Test	How to Test	Pass Criteria
Unauthenticated access	Call any API endpoint without token	Returns 401 Unauthorized
Wrong role access	Log in as Client_Reviewer, call admin endpoints	Returns 403 Forbidden
Cross-tenant access	Log in as User A, access User B project ID	Returns 403 Forbidden
File type bypass	Upload .exe renamed as .mp4	Rejected with 400 Bad Request
SQL injection	Submit '--; DROP TABLE users in text fields	No DB error; input safely parameterised
Oversized file	Upload file > 1GB (configured limit)	Rejected at upload with 413 error
Expired JWT	Use a token past its expiry time	Returns 401 Unauthorized

Phase 7 — Docker Compose Production-Ready Deployment (Week 5)

Before any cloud migration, deploy the full system using Docker Compose on a simple VPS (e.g. a DigitalOcean Droplet or any Linux server). This validates the system end-to-end under real conditions.

Step 7.1 — Dockerise the Backend

```
# backend/Dockerfile
FROM python:3.12-slim

RUN apt-get update && apt-get install -y ffmpeg libpq-dev gcc && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 7.2 — Dockerise the Frontend

```
# frontend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json .
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Step 7.3 — Production Docker Compose

```
# docker-compose.prod.yml
version: "3.9"
services:

  api:
    build: ./backend
    env_file: .env.prod
    depends_on: [postgres, redis, minio]
    ports: ["8000:8000"]
    restart: unless-stopped

  worker:
    build: ./backend
    command: celery -A workers.celery_app worker -Q transcription,extraction,prd -c 4
    env_file: .env.prod
    depends_on: [postgres, redis, minio]
    restart: unless-stopped
```

```
  frontend:
    build: ./frontend
    ports: ["80:80"]
    restart: unless-stopped

  postgres:
    image: pgvector/pgvector:pg15
    env_file: .env.prod
    volumes: ["postgres_data:/var/lib/postgresql/data"]
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    env_file: .env.prod
    volumes: ["minio_data:/data"]
    ports: ["9000:9000"]
    restart: unless-stopped

volumes:
  postgres_data:
  minio_data:
```

Step 7.4 — Production Launch Checklist

Item	Status	Owner
SECRET_KEY set to a strong random 32-char value	[ ] Done	Backend
All .env.prod values filled in (no placeholder values)	[ ] Done	DevOps
SSL certificate configured (Let's Encrypt + Nginx)	[ ] Done	DevOps
PostgreSQL volume backed up before every deployment	[ ] Done	DevOps
MinIO storage bucket set to private (no public access)	[ ] Done	DevOps
Rate limiting middleware enabled on all API routes	[ ] Done	Backend
GDPR data-deletion endpoint tested end-to-end	[ ] Done	Backend
Audit log table verified (all access events recorded)	[ ] Done	Backend
Internal UAT session completed with 5+ real users	[ ] Done	Product
Client portal tested end-to-end with a dummy client	[ ] Done	QA
Load test passed (50 concurrent users, no degradation)	[ ] Done	QA

Step 7.5 — Monitoring (Simple, No GCP Required)

- Add /api/health endpoint — return DB connection status, Redis ping, MinIO ping
- Use UptimeRobot (free) to ping /api/health every 5 minutes and alert via email/Slack

- Add Sentry SDK to the FastAPI app for error tracking (free tier covers development)
- Use Celery Flower (included with Celery) for task queue monitoring: flower -A workers.celery_app
- PostgreSQL logs: docker compose logs postgres -f

> **Tip: At this stage you have a fully working system on a simple VPS.**
> Only proceed to Phase 8 (GCP migration) once the system is validated,
> there is a clear need for auto-scaling or managed services, and the
> team has the capacity to manage cloud infrastructure.

Phase 8 — GCP Migration (Production Scale)

This phase is optional during initial development. Follow it when the system is proven and you need managed infrastructure, auto-scaling, or enterprise-level reliability.

> Prerequisites before starting Phase 8:
> 1. All Phase 1–7 tests pass consistently
> 2. At least one real client has used the system successfully
> 3. You have a GCP account with billing enabled
> 4. The DevOps engineer has completed GCP training (Cloud Run, Cloud SQL, Pub/Sub)

Step 8.1 — What Changes vs. What Stays the Same

Component	Dev Version (Phases 1–7)	GCP Version (Phase 8)	Effort
Database	PostgreSQL in Docker	Cloud SQL (PostgreSQL 15)	Config only
File Storage	MinIO in Docker	GCP Cloud Storage	Swap boto3 endpoint URL
Message Queue	Celery + Redis	Google Cloud Pub/Sub	Rewrite worker trigger
Auth	FastAPI JWT	Firebase Authentication	Add Firebase SDK, remove JWT
AI	Gemini API (direct)	Vertex AI (Gemini 1.5 Pro)	Swap genai SDK for vertexai
Backend Hosting	Docker on VPS	Cloud Run (auto-scaling)	Add Dockerfile + cloudbuild.yaml
Frontend Hosting	Nginx on VPS	Cloud CDN + Cloud Run	Update deploy script
Secrets	.env file	GCP Secret Manager	Update config.py to read from SM
Background Workers	Celery + Redis in Docker	Cloud Run + Pub/Sub	Rewrite task dispatch

Step 8.2 — GCP Project Setup

1. Go to console.cloud.google.com — create project: xccelera-prd-saas

2. Enable billing on the project.
3. Enable APIs: Cloud Run, Cloud SQL Admin, Cloud Storage, Pub/Sub, Vertex AI, Secret Manager, Cloud Build, Artifact Registry, Firebase
4. Install and configure: gcloud auth login && gcloud config set project YOUR_PROJECT_ID

Step 8.3 — Migrate Database to Cloud SQL

```
# Create Cloud SQL instance (Terraform or gcloud)
gcloud sql instances create prd-postgres \
  --database-version=POSTGRES_15 \
  --tier=db-g1-small \
  --region=asia-south1 \
  --database-flags=cloudsql.enable_pgvector=on

# Export local data
pg_dump postgresql://prd_user:prd_pass@localhost:5432/prd_db > backup.sql

# Import to Cloud SQL (via Cloud SQL proxy)
cloud-sql-proxy xccelera-prd-saas:asia-south1:prd-postgres &
psql "host=127.0.0.1 user=prd_user dbname=prd_db" < backup.sql

# Update DATABASE_URL in Secret Manager:
# postgresql+asyncpg://user:pass@/prd_db?host=/cloudsql/project:region:instance
```

Step 8.4 — Migrate File Storage to Cloud Storage

The storage service uses the boto3 S3-compatible API. Switching from MinIO to GCS is a single environment variable change:

```
# services/storage.py — no code change needed
# Just update .env / Secret Manager:

# Dev (MinIO):
STORAGE_ENDPOINT=http://localhost:9000
STORAGE_ACCESS_KEY=minioadmin
STORAGE_SECRET_KEY=minioadmin

# Production (GCS via HMAC keys):
STORAGE_ENDPOINT=https://storage.googleapis.com
STORAGE_ACCESS_KEY=your_gcs_hmac_access_key
STORAGE_SECRET_KEY=your_gcs_hmac_secret
STORAGE_BUCKET=xccelera-prd-uploads

# Create buckets in GCS:
gsutil mb -l asia-south1 gs://xccelera-prd-uploads
gsutil mb -l asia-south1 gs://xccelera-prd-exports
```

Step 8.5 — Replace Celery with Pub/Sub

This is the most involved migration. The worker logic stays the same — only the trigger mechanism changes from Celery tasks to Pub/Sub messages:

Celery Task (Dev)	Pub/Sub Equivalent (GCP)	Notes
transcribe_file.delay(id)	publish to file-uploaded topic	Worker subscribes via Cloud Run HTTP push

Celery Task (Dev)	Pub/Sub Equivalent (GCP)	Notes
extract_requirements.delay(id)	publish to transcription-done topic	Same Cloud Run service, different endpoint
generate_prd.delay(id)	publish to extraction-done topic	Triggered when all source files are done

```
# services/queue.py — create a simple abstraction
import os

MODE = os.getenv('QUEUE_MODE', 'celery')  # 'celery' | 'pubsub'

def enqueue_transcription(source_file_id: str):
  if MODE == 'celery':
    from workers.tasks import transcribe_file
    transcribe_file.delay(source_file_id)
  else:
    from app.services.pubsub import publish
    publish('file-uploaded', {'source_file_id': source_file_id})
```

Step 8.6 — Deploy Backend to Cloud Run

```
# cloudbuild.yaml
steps:
  - name: gcr.io/cloud-builders/docker
    args: ["build", "-t", "gcr.io/$PROJECT_ID/prd-api", "./backend"]
  - name: gcr.io/cloud-builders/docker
    args: ["push", "gcr.io/$PROJECT_ID/prd-api"]
  - name: gcr.io/cloud-builders/gcloud
    args:
      - run
      - deploy
      - prd-api
      - --image=gcr.io/$PROJECT_ID/prd-api
      - --region=asia-south1
      - --platform=managed
      - --min-instances=1
      - --max-instances=10
      - --set-secrets=DATABASE_URL=DATABASE_URL:latest
```

Step 8.7 — Replace JWT Auth with Firebase

Firebase Authentication replaces the simple JWT system. The main change is in the token verification middleware:

```
# Before (dev JWT):
def get_current_user(token: str = Depends(oauth2_scheme)):
  return verify_token(token)  # jose.jwt.decode()

# After (Firebase):
import firebase_admin.auth as fb_auth

def get_current_user(token: str = Depends(oauth2_scheme)):
  decoded = fb_auth.verify_id_token(token)
  return {'user_id': decoded['uid'], 'role': decoded.get('role', 'BA_PM')}

# Frontend: replace axios JWT header with Firebase ID token:
# const token = await firebase.auth().currentUser.getIdToken();
```

```
# axios.defaults.headers.Authorization = `Bearer ${token}`;
```

Step 8.8 — GCP Migration Checklist

Item	Status	Owner
Cloud SQL instance running with pgvector extension	[ ] Done	DevOps
Data migrated from local PostgreSQL to Cloud SQL	[ ] Done	DevOps
GCS buckets created; files migrated from MinIO	[ ] Done	DevOps
Pub/Sub topics and subscriptions created	[ ] Done	DevOps
All secrets moved to GCP Secret Manager	[ ] Done	DevOps
Cloud Run services deployed (API + workers)	[ ] Done	DevOps
Custom domain + SSL configured via Cloud Load Balancer	[ ] Done	DevOps
Firebase Auth configured; JWT auth removed	[ ] Done	Backend
Vertex AI replacing direct Gemini API calls	[ ] Done	Backend / ML
CI/CD pipeline updated (GitHub Actions → Cloud Build)	[ ] Done	DevOps
All Phase 6 tests re-run against GCP environment	[ ] Done	QA
Load test re-run: 50 concurrent users on Cloud Run	[ ] Done	QA

> **Tip: Run Phase 6 tests again after migration — each component swap is a potential regression.**
> Keep the Docker Compose stack running in parallel until all GCP tests pass.
> Do NOT decommission the VPS until you have 1 week of stable GCP production traffic.

Appendix A — Environment Variables Reference

All values shown are for development. Phase 8 substitutes the GCP equivalents.

Variable	Dev Value	Production (GCP) Value
DATABASE_URL	postgresql+asyncpg://prd_user:prd_pass@localhost:5432/prd_db	Cloud SQL socket connection string
REDIS_URL	redis://localhost:6379/0	Redis Cloud or Cloud Memorystore URL
STORAGE_ENDPOINT	http://localhost:9000	https://storage.googleapis.com
STORAGE_ACCESS_KEY	minioadmin	GCS HMAC access key
STORAGE_SECRET_KEY	minioadmin	GCS HMAC secret key
STORAGE_BUCKET	prd-uploads	xccelera-prd-uploads
GEMINI_API_KEY	AI Studio API key	Not needed — use Vertex AI service account
SECRET_KEY	dev-secret-32chars	Strong random — via Secret Manager
SMTP_HOST	sandbox.smtp.mailtrap.io	smtp.sendgrid.net
SMTP_PORT	2525	587
QUEUE_MODE	celery	pubsub

Appendix B — Tech Stack Summary

Layer	Dev Technology	GCP Production Equivalent
Backend API	FastAPI 0.111 + Python 3.12	Same — deployed on Cloud Run
Transcription	Whisper (local Python library)	Whisper on Cloud Run (containerised)
AI / LLM	Gemini API via google-generativeai	Vertex AI (gemini-1.5-pro)
LLM Fallback	OpenAI API	Claude / GPT-4 via Secret Manager keys
Database	PostgreSQL 15 + pgvector (Docker)	Cloud SQL PostgreSQL 15
File Storage	MinIO (Docker, S3-compatible)	GCP Cloud Storage
Message Queue	Celery + Redis (Docker)	Google Cloud Pub/Sub
Auth	FastAPI JWT (jose + bcrypt)	Firebase Authentication

Layer	Dev Technology	GCP Production Equivalent
Frontend	React 18 + Tailwind CSS + Vite	Same — static files on Cloud CDN
Email	Mailtrap (dev sandbox)	SendGrid
Infrastructure	Docker Compose	Terraform on GCP
CI/CD	GitHub Actions (docker build + push)	GitHub Actions + Cloud Build
Monitoring	UptimeRobot + Sentry + Flower	Cloud Monitoring + Cloud Logging

End of Document — AI-Powered PRD System Build Guide v2.0 (Development-First Edition)